

Résumé Complet Cypher Neo4j

CRUD, Requêtes, Relations

AmineGR03

1^{er} février 2026

Table des matières

Opérations CRUD

Create (Création)

Créer un nœud

```
1 CREATE (p:Person {nom: 'Dupont', age: 30, ville: 'Paris'})
2 RETURN p;
```

Créer plusieurs nœuds

```
1 CREATE
2   (p1:Person {nom: 'Dupont', age: 30}),
3   (p2:Person {nom: 'Martin', age: 25}),
4   (p3:Person {nom: 'Bernard', age: 35})
5 RETURN p1, p2, p3;
```

Créer une relation

```
1 CREATE (p1:Person {nom: 'Dupont'})-[:CONNAIT]->(p2:Person {nom: 'Martin'})
2 RETURN p1, p2;
```

Créer une relation entre nœuds existants

```
1 MATCH (p1:Person {nom: 'Dupont'}), (p2:Person {nom: 'Martin'})
2 CREATE (p1)-[:CONNAIT {depuis: 2020}]->(p2)
3 RETURN p1, p2;
```

Créer un chemin complet

```
1 CREATE path = (a:Person {nom: 'Alice'})-[:CONNAIT]->(b:Person {nom: 'Bob'})
2   -[:CONNAIT]->(c:Person {nom: 'Charlie'})
3 RETURN path;
```

MERGE (Créer ou Mettre à jour)

MERGE simple

```
1 MERGE (p:Person {nom: 'Dupont'})
2 RETURN p;
```

MERGE avec SET

```
1 MERGE (p:Person {nom: 'Dupont'})
2 ON CREATE SET p.age = 30, p.ville = 'Paris'
3 ON MATCH SET p.age = 31
4 RETURN p;
```

MERGE pour relation

```
1 MATCH (p1:Person {nom: 'Dupont'}), (p2:Person {nom: 'Martin'})
2 MERGE (p1)-[r:CONNAIT]->(p2)
3 ON CREATE SET r.depuis = 2020
4 ON MATCH SET r.depuis = 2020
5 RETURN r;
```

Read (Lecture)

MATCH simple

```
1 MATCH (p:Person)
2 RETURN p;
```

MATCH avec filtre

```
1 MATCH (p:Person {nom: 'Dupont'})
2 RETURN p;
```

MATCH avec WHERE

```
1 MATCH (p:Person)
2 WHERE p.age > 25
3 RETURN p;
```

MATCH avec relations

```
1 MATCH (p1:Person)-[:CONNAIT]->(p2:Person)
2 RETURN p1, p2;
```

MATCH avec direction

```
1 // Relation sortante
2 MATCH (p:Person)-[:CONNAIT]->(autre:Person)
3 RETURN p, autre;
4
5 // Relation entrante
6 MATCH (p:Person)<-[:CONNAIT]-(autre:Person)
7 RETURN p, autre;
8
9 // Relation bidirectionnelle
10 MATCH (p:Person)-[:CONNAIT]-(autre:Person)
11 RETURN p, autre;
```

MATCH avec profondeur

```
1 // 1 niveau
2 MATCH (p:Person)-[:CONNAIT]->(ami:Person)
3 RETURN p, ami;
4
5 // 2 niveaux
6 MATCH (p:Person)-[:CONNAIT]->(ami:Person)-[:CONNAIT]->(amiDeAmi:Person)
7 RETURN p, ami, amiDeAmi;
8
9 // Profondeur variable
10 MATCH (p:Person)-[:CONNAIT*1..3]->(autre:Person)
11 RETURN p, autre;
```

OPTIONAL MATCH

```
1 MATCH (p:Person)
2 OPTIONAL MATCH (p)-[:CONNAIT]->(ami:Person)
3 RETURN p, ami;
```

Update (Mise à jour)

SET simple

```
1 MATCH (p:Person {nom: 'Dupont'})
2 SET p.age = 31
3 RETURN p;
```

SET multiple

```
1 MATCH (p:Person {nom: 'Dupont'})
2 SET p.age = 31, p.ville = 'Lyon'
3 RETURN p;
```

SET avec propriété calculée

```
1 MATCH (p:Person)
2 SET p.agePlusUn = p.age + 1
3 RETURN p;
```

Ajouter un label

```
1 MATCH (p:Person {nom: 'Dupont'})
2 SET p:Client
3 RETURN p;
```

Ajouter plusieurs labels

```
1 MATCH (p:Person {nom: 'Dupont'})
2 SET p:Client:VIP
3 RETURN p;
```

Mettre à jour une relation

```
1 MATCH (p1:Person {nom: 'Dupont'})-[r:CONNAIT]->(p2:Person {nom: 'Martin'})
2 SET r.depuis = 2021
3 RETURN r;
```

Delete (Suppression)

DELETE nœud

```
1 MATCH (p:Person {nom: 'Dupont'})
2 DELETE p;
```

DELETE relation

```
1 MATCH (p1:Person {nom: 'Dupont'})-[r:CONNAIT]->(p2:Person {nom: 'Martin'})
2 DELETE r;
```

DELETE multiple

```
1 MATCH (p:Person)
2 WHERE p.age < 18
3 DELETE p;
```

DETACH DELETE (supprime nœud et relations)

```
1 MATCH (p:Person {nom: 'Dupont'})
2 DETACH DELETE p;
```

REMOVE (supprimer propriété ou label)

```
1 // Supprimer une propriété
2 MATCH (p:Person {nom: 'Dupont'})
3 REMOVE p.ville
4 RETURN p;
5
6 // Supprimer un label
7 MATCH (p:Person {nom: 'Dupont'})
8 REMOVE p:Client
9 RETURN p;
```

Filtrage et Conditions

Opérateurs de Comparaison

```
1 // galit
2 MATCH (p:Person)
3 WHERE p.age = 30
4 RETURN p;
5
6 // Plus grand que
7 WHERE p.age > 25
8
9 // Plus grand ou gal
10 WHERE p.age >= 25
11
12 // Plus petit que
13 WHERE p.age < 40
14
15 // Plus petit ou gal
16 WHERE p.age <= 40
17
18 // Différent de
19 WHERE p.age <> 30
```

Opérateurs Logiques

```
1 // ET
2 MATCH (p:Person)
3 WHERE p.age > 25 AND p.ville = 'Paris'
4 RETURN p;
5
6 // OU
7 WHERE p.age < 30 OR p.ville = 'Lyon'
8
9 // NON
10 WHERE NOT p.ville = 'Paris'
```

Opérateurs pour Chaînes

```
1 // Contient
2 WHERE p.nom CONTAINS 'Dup'
3
4 // Commence par
5 WHERE p.nom STARTS WITH 'Dup'
6
7 // Se termine par
8 WHERE p.nom ENDS WITH 'ont'
9
10 // Expression régulière
11 WHERE p.nom =~ 'Dup.*'
```

Opérateur IN

```
1 MATCH (p:Person)
2 WHERE p.ville IN ['Paris', 'Lyon', 'Marseille']
3 RETURN p;
```

Opérateur EXISTS

```
1 MATCH (p:Person)
2 WHERE EXISTS(p.email)
3 RETURN p;
```

Opérateur IS NULL

```
1 MATCH (p:Person)
2 WHERE p.email IS NULL
3 RETURN p;
```

Fonctions et Agrégation

Fonctions sur Nœuds

```
1 // ID du nœud
2 MATCH (p:Person)
3 RETURN id(p), p;
4
5 // Labels du nœud
6 RETURN labels(p);
7
8 // Propriétés du nœud
9 RETURN properties(p);
10
11 // Clés des propriétés
12 RETURN keys(p);
```

Fonctions sur Relations

```
1 MATCH (p1:Person)-[r:CONNAIT]->(p2:Person)
2 RETURN
3     type(r) AS typeRelation,
4     id(r) AS idRelation,
5     properties(r) AS propsRelation;
```

Fonctions d'Agrégation

```
1 // COUNT
2 MATCH (p:Person)
3 RETURN COUNT(p) AS nombrePersonnes;
4
5 // COUNT DISTINCT
6 RETURN COUNT(DISTINCT p.ville) AS nombreVilles;
7
8 // SUM
9 MATCH (p:Person)
10 RETURN SUM(p.age) AS sommeAges;
11
12 // AVG
13 RETURN AVG(p.age) AS ageMoyen;
14
15 // MIN
16 RETURN MIN(p.age) AS ageMin;
17
18 // MAX
19 RETURN MAX(p.age) AS ageMax;
20
21 // COLLECT
22 RETURN COLLECT(p.nom) AS tousLesNoms;
```

GROUP BY


```
1 MATCH (p:Person)
2 RETURN p.ville, COUNT(p) AS nombre
3 ORDER BY nombre DESC;
```

Requêtes Avancées

WITH (Pipeline)

```
1 MATCH (p:Person)
2 WITH p
3 WHERE p.age > 25
4 RETURN p.nom, p.age
5 ORDER BY p.age DESC
6 LIMIT 10;
```

UNWIND

```
1 UNWIND ['Paris', 'Lyon', 'Marseille'] AS ville
2 MATCH (p:Person {ville: ville})
3 RETURN ville, COLLECT(p.nom) AS personnes;
```

UNION

```
1 MATCH (p:Person {ville: 'Paris'})
2 RETURN p.nom AS nom
3 UNION
4 MATCH (p:Person {ville: 'Lyon'})
5 RETURN p.nom AS nom;
```

UNION ALL

```
1 MATCH (p:Person {ville: 'Paris'})
2 RETURN p.nom AS nom
3 UNION ALL
4 MATCH (p:Person {ville: 'Lyon'})
5 RETURN p.nom AS nom;
```

CASE WHEN

```
1 MATCH (p:Person)
2 RETURN
3     p.nom,
4     CASE
5         WHEN p.age < 30 THEN 'Jeune'
6         WHEN p.age < 50 THEN 'Adulte'
7         ELSE 'Senior'
8     END AS categorie;
```

FOREACH

```
1 MATCH (p:Person)
2 FOREACH (nom IN ['Dupont', 'Martin'] |
3     SET p.estImportant = true
4 );
```

Chemins et Traversée

Chemins Variables

```
1 MATCH path = (p1:Person)-[:CONNAIT*]->(p2:Person)
2 RETURN path;
```

SHORTESTPATH

```
1 MATCH path = shortestPath(
2     (p1:Person {nom: 'Dupont'})-[*]-(p2:Person {nom: 'Martin'})
3 )
4 RETURN path;
```

ALLSHORTESTPATHS

```
1 MATCH paths = allShortestPaths(
2     (p1:Person {nom: 'Dupont'})-[*]-(p2:Person {nom: 'Martin'})
3 )
4 RETURN paths;
```

Profondeur Variable

```
1 // 1      3 niveaux
2 MATCH (p:Person)-[:CONNAIT*1..3]->(autre:Person)
3 RETURN p, autre;
4
5 // Exactement 2 niveaux
6 MATCH (p:Person)-[:CONNAIT*2]->(autre:Person)
7 RETURN p, autre;
8
9 // 2 niveaux ou plus
10 MATCH (p:Person)-[:CONNAIT*2..]->(autre:Person)
11 RETURN p, autre;
```

Relations Multiples

```
1 MATCH (p:Person)-[:CONNAIT|AIME|TRAVAILLE_AVEC]->(autre:Person)
2 RETURN p, autre;
```

Index et Contraintes

Créer un Index

```
1 CREATE INDEX ON :Person(nom);  
2 CREATE INDEX ON :Person(age);
```

Index Composite

```
1 CREATE INDEX ON :Person(nom, age);
```

Contrainte Unique

```
1 CREATE CONSTRAINT ON (p:Person) ASSERT p.email IS UNIQUE;
```

Contrainte d'Existence

```
1 CREATE CONSTRAINT ON (p:Person) ASSERT EXISTS(p.nom);
```

Supprimer Index

```
1 DROP INDEX ON :Person(nom);
```

Supprimer Contrainte

```
1 DROP CONSTRAINT ON (p:Person) ASSERT p.email IS UNIQUE;
```

Exemples Pratiques

Exemple 1 : Recommandations d'Amis

```
1 MATCH (moi:Person {nom: 'Dupont'})-[:CONNAIT]->(ami:Person)-[:CONNAIT  
  ]->(amiDeAmi:Person)  
2 WHERE NOT (moi)-[:CONNAIT]->(amiDeAmi)  
3 AND moi <> amiDeAmi  
4 RETURN DISTINCT amiDeAmi.nom AS recommandation  
5 LIMIT 10;
```

Exemple 2 : Plus Court Chemin

```
1 MATCH path = shortestPath(  
2   (start:Person {nom: 'Dupont'})-[*]-(end:Person {nom: 'Martin'})  
3 )  
4 RETURN path, length(path) AS distance;
```

Exemple 3 : Statistiques par Ville

```
1 MATCH (p:Person)  
2 RETURN  
3   p.ville AS ville,  
4   COUNT(p) AS nombre,  
5   AVG(p.age) AS ageMoyen,  
6   MIN(p.age) AS ageMin,  
7   MAX(p.age) AS ageMax  
8 ORDER BY nombre DESC;
```

Exemple 4 : Communauté (Personnes Connectées)

```
1 MATCH (p:Person {nom: 'Dupont'})-[:CONNAIT*1..2]-(autre:Person)  
2 RETURN DISTINCT autre.nom AS dansLaCommunaute  
3 ORDER BY autre.nom;
```

Exemple 5 : Relations les Plus Fréquentes

```
1 MATCH ()-[r:CONNAIT]->()  
2 RETURN type(r) AS typeRelation, COUNT(r) AS frequence  
3 ORDER BY frequence DESC;
```

Résumé des Opérations Essentielles

CRUD

Opération	Commande
Create	CREATE, MERGE
Read	MATCH ... RETURN
Update	MATCH ... SET
Delete	MATCH ... DELETE, DETACH DELETE
Remove	MATCH ... REMOVE

Opérateurs de Filtrage

Opérateur	Description
=	Égalité
>, >=	Comparaison
<, <=	Comparaison
<>	Différent de
AND, OR, NOT	Logique
CONTAINS	Contient
STARTS WITH	Commence par
ENDS WITH	Se termine par
IN	Dans une liste
EXISTS	Existe
IS NULL	Est nul

Fonctions d'Agrégation

Fonction	Description
COUNT()	Compter
SUM()	Somme
AVG()	Moyenne
MIN()	Minimum
MAX()	Maximum
COLLECT()	Collecter en tableau

Points Importants

- **MATCH** : Trouve des patterns dans le graphe
- **CREATE** : Crée toujours (peut créer des doublons)
- **MERGE** : Crée si n'existe pas, trouve si existe
- **Relations** : Utilisent -[:TYPE]-> pour direction
- **Profondeur variable** : Utilise [*1..3] pour 1 à 3 niveaux
- **OPTIONAL MATCH** : Permet des résultats même si pas de match
- **DETACH DELETE** : Supprime nœud et toutes ses relations